

ISOLATION FOREST

Liu, Ting & Zhou | ICDM 2008

TL;DR

Isolation Forest is an anomaly detection algorithm that finds outliers by isolating them with random splits in a tree — anomalies are isolated faster (shorter paths) because they are few and far from normal data.

1. The Core Problem with Traditional Anomaly Detection

Most anomaly detection methods work by first building a detailed picture of what 'normal' looks like, and then flagging anything that doesn't fit. Think of it like memorising every face in your school so you can spot a stranger.

This approach has two big problems:

- It is optimised for modelling normal data, not for catching anomalies — the two goals are not the same.
- It is computationally expensive and struggles with large or high-dimensional datasets.

KEY INSIGHT

Isolation Forest flips the problem: instead of profiling what is normal, it directly isolates what is abnormal.

2. The Core Idea — Anomalies Are 'Few and Different'

The authors observe that anomalies share two properties that make them easy to isolate:

- They are the minority — there are very few of them.
- Their attribute values are very different from normal points.

Because of these two properties, you can isolate an anomaly with just a few random splits of your data, while a normal point requires many more splits to separate it from its neighbours.

ANALOGY

Imagine the data as a crowd. An anomaly is like a person standing alone on the edge — one rope barrier isolates them. A normal person in the middle of the crowd needs many barriers to be isolated.

3. How It Works — Step by Step

Step 1: Build an Isolation Tree (iTree)

Take a random sub-sample of your data. Then, repeatedly split the data by:

- Randomly picking one feature (attribute).
- Randomly picking a split value between the min and max of that feature.
- Splitting data into left ($<$ split) and right ($\geq$ split) branches.

Keep splitting until every data point is in its own leaf node (isolated), or until the tree reaches a maximum height. The result is a binary tree.

Step 2: Measure Path Length

For any data point, count the number of edges it travels from the root to reach its leaf. This is the path length $h(x)$.

- Short path length $\rightarrow$ isolated quickly $\rightarrow$ likely an anomaly.
- Long path length $\rightarrow$ took many splits to isolate $\rightarrow$ likely normal.

Step 3: Build a Forest of iTrees

A single tree is too noisy. So the algorithm builds t trees (default: 100), each trained on a different random sub-sample. The expected (average) path length $E(h(x))$ over all trees is used for the final score.

Step 4: Compute the Anomaly Score

The raw path length is normalised against $c(n)$, the expected average path length for a dataset of size n (borrowed from Binary Search Tree theory):

$$s(x, n) = 2^{(-E(h(x)) / c(n))}$$

Score (s)	Interpretation
$s \rightarrow 1.0$	Definitely an anomaly — very short path
$s > 0.6$	Likely anomaly — worth investigating
$s \approx 0.5$	Normal — path length near average
$s < 0.5$	Clearly normal — deep in the tree

4. Handling Swamping and Masking

Two classic failure modes in anomaly detection are:

- Swamping — falsely labelling normal points as anomalies (because they are close to real anomalies).
- Masking — anomalies concealing each other when there is a large, dense anomaly cluster.

Isolation Forest deals with both by using a small sub-sample size. When you use only ~128–256 points per tree:

- The noise of nearby normal points surrounding an anomaly is reduced.
- Dense anomaly clusters are broken up — individual trees see smaller slices of the cluster.

EMPIRICAL RESULT

On the Mulcross dataset: using the full 4,096 points → AUC = 0.67. Using a sub-sample of 128 → AUC = 0.91.

5. Key Parameters

There are only two parameters to tune:

Parameter	Default Value	Notes
ψ (sub-sample size)	256	Bigger is NOT always better. AUC converges quickly at small values.
t (number of trees)	100	Path lengths converge well before 100 trees in most datasets.

6. Complexity and Efficiency

One of the strongest selling points of Isolation Forest is its computational profile:

Stage	Complexity	Variables
Training	$O(t \cdot \psi \cdot \log \psi)$	t = trees, ψ = sub-sample size
Evaluation	$O(n \cdot t \cdot \log \psi)$	n = test set size

With $\psi = 256$ and $t = 100$, a model uses at most $511 \times 100 = 51,100$ nodes — trivial memory even on a laptop.

7. Comparison with Other Methods

Method	Complexity	Memory	Large Data	No Distance?
iForest	$O(n)$ linear	Low	Excellent	Yes
LOF	$O(n^2)$	High	Poor	No
ORCA (k-NN)	$\sim O(n)$	High	Good	No
Random Forest	$O(n \log n)$	Very High	Poor	No

Empirical results on 12 real-world datasets (KDD CUP99, ForestCover, Shuttle, etc.) showed iForest outperforming ORCA in AUC on most datasets, being orders of magnitude faster on large data (e.g., 15 seconds vs. 9,487 seconds on HTTP dataset), and handling the masking/swamping problem that defeats distance-based methods.

8. Practical Strengths and Limitations

Strengths

- Works without labelled anomalies — fully unsupervised.
- Scales linearly to millions of data points.
- Handles high-dimensional data (with optional Kurtosis-based attribute selection).
- Naturally handles the swamping and masking problem via sub-sampling.
- Very low memory footprint.

Limitations

- Struggles with datasets where anomalies are not globally rare (e.g., 35% anomaly rate in Pima/Ionosphere — AUC drops to 0.67–0.85).
- Random splits are axis-aligned — cannot detect anomalies that are only unusual in the interaction of features (Extended Isolation Forest addresses this).
- Assumes anomalies are globally few and different — does not handle local anomalies well.

9. Summary

Isolation Forest is an elegant and efficient anomaly detection algorithm that exploits a simple insight: anomalies are easier to isolate than normal points. By building ensembles of random trees on small sub-samples, it achieves linear time complexity with very low memory usage, making it practical for large, real-world datasets.

Its key innovations are: (1) isolation-first philosophy instead of profiling normal data, (2) sub-sampling that reduces swamping and masking, and (3) a normalised anomaly score derived from average path length. These make it an ideal baseline for any anomaly detection pipeline before deploying more sophisticated temporal models.